BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 7

Roll no: 44 PRN No: 12410016 Name of Student: ***Sahil Tagade***

Subject Name : Deep Learning

1. Aim

To implement and compare image-classification models using transfer learning with VGG16, ResNet50 and EfficientNet-B0, along with an AlexNet-style CNN baseline, using TensorFlow/Keras.

2. Objectives

Understand transfer learning using ImageNet-pretrained convolutional neural networks.
Use a small Cat-vs-Dog subset of CIFAR-10 to reduce memory and execution time.
Freeze the pretrained feature extraction layers and train a new binary classifier.
Implement VGG16, ResNet50 and EfficientNet-B0 using TensorFlow/Keras.
Implement an AlexNet-style CNN baseline because official pretrained AlexNet weights are not included in tf.keras.applications.
Compare Accuracy, Precision, Recall and F1-Score.

3. Dataset

CIFAR-10 is used for the experiment. It contains 10 image categories. Only Cat and Dog are selected for binary classification. The experiment uses 500 images from each class for training and 100 images from each class for validation. Images are resized from 32×32 to 128×128 pixels.

Classes: Cat and Dog
Training images: 1,000
Validation images: 200
Input size: 128×128×3
Batch size: 8
Epochs: 2

4. Transfer Learning Theory

Transfer learning reuses knowledge learned by a neural network on a large source dataset for a different target task. Here, VGG16, ResNet50 and EfficientNet-B0 are loaded with ImageNet pretrained weights. Their convolutional base is frozen, and a new binary classification layer is added for Cat/Dog classification.

Freezing the base reduces the number of trainable parameters and makes the experiment faster and more memory efficient. The same dataset, preprocessing and evaluation metrics are used for all models to make the comparison fair.

5. Important AlexNet Note

TensorFlow/Keras doesnot provide an official ImageNet-pretrained AlexNet model through tf.keras.applications. Therefore, this assignment implements an AlexNet-style architecture as a baseline rather than falsely claiming that pretrained AlexNet weights are available. VGG16, ResNet50 and EfficientNet-B0 use official ImageNet pretrained weights. This distinction should be mentioned during viva if asked.

If the college specifically mandates pretrained AlexNet weights, an approved converted AlexNet checkpoint must be supplied and loaded separately. The rest of the experiment remains unchanged.

6. Models Used

Model	Type	Main idea
AlexNet	AlexNet-style baseline	Five convolutional stages followed by classification
VGG16	ImageNet pretrained	Stacked 3×3 convolution layers
ResNet50	ImageNet pretrained	Residual/skip connections
EfficientNet-B0	ImageNet pretrained	Efficient compound scaling

7. Methodology

Load CIFAR-10 using keras.datasets.cifar10.
Select Cat and Dog classes only.
Resize all images to 128×128.
Create small TensorFlow tf.data input pipelines.
Load VGG16, ResNet50 and EfficientNet-B0 with weights='imagenet'.
Freeze their base layers.
Add Global Average Pooling, Dropout and a sigmoid output layer.
Train using Adam optimizer and binary cross-entropy loss.
Evaluate using Accuracy, Precision, Recall and F1-Score.
Run models sequentially and clear the Keras session after each model to reduce memory usage.

8. Key Implementation

```
base =VGG16(include_top=False, weights="imagenet", input_shape=(128,128,3))
base.trainable = False
x = base(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
```

9. Evaluation Metrics

Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1-Score = 2 × Precision × Recall / (Precision + Recall)

Accuracy measures overall correctness. Precision measures the correctness of positive predictions. Recall measures how many actual positive samples are detected. F1-Score balances Precision and Recall.

10. Results

Runthe supplied TensorFlow program and enter the actual values printed by the program. Do not use fabricated values because the exact results depend on the software version and run.

1) ***AlexNet***

```
============================================================
Running: AlexNet
C:\Sem5-(TY)\Deep Learning\Deep Learning Labs\Assignment7\venv\Lib\site-packages\keras\src\trainers\epoch_iterator.py:74:
UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Data
set is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
Epoch 1/2
125/125 ━━━━━━━━━━━━━━━━━━━━ 14s 88ms/step - accuracy: 0.4850 - loss: 0.7108 - val_accuracy: 0.5000 - val_loss: 0.6931
Epoch 2/2
125/125 ━━━━━━━━━━━━━━━━━━━━ 11s 89ms/step - accuracy: 0.4720 - loss: 0.6938 - val_accuracy: 0.5000 - val_loss: 0.6932

Final metrics:
Model: AlexNet
Accuracy: 0.5000
Precision: 0.5000
Recall: 1.0000
F1-Score: 0.6667

Classification Report:
              precision    recall  f1-score   support

         Cat       0.00      0.00      0.00       100
         Dog       0.50      1.00      0.67       100

    accuracy                           0.50       200
   macro avg       0.25      0.50      0.33       200
weighted avg       0.25      0.50      0.33       200
```

2) ***VGG16***

```
============================================================
Running: VGG16
C:\Sem5-(TY)\Deep Learning\Deep Learning Labs\Assignment7\venv\Lib\site-packages\keras\src\trainers\epoch_iterator.py:74:
UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Data
set is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
Epoch 1/2
125/125 ━━━━━━━━━━━━━━━━━━━━ 87s 683ms/step - accuracy: 0.5390 - loss: 2.1974 - val_accuracy: 0.5850 - val_loss: 1.6180
Epoch 2/2
125/125 ━━━━━━━━━━━━━━━━━━━━ 87s 696ms/step - accuracy: 0.6570 - loss: 1.3154 - val_accuracy: 0.6600 - val_loss: 1.2507

Final metrics:
Model: VGG16
Accuracy: 0.6600
Precision: 0.6702
Recall: 0.6300
F1-Score: 0.6495

Classification Report:
              precision    recall  f1-score   support

         Cat       0.65      0.69      0.67       100
         Dog       0.67      0.63      0.65       100

    accuracy                           0.66       200
   macro avg       0.66      0.66      0.66       200
weighted avg       0.66      0.66      0.66       200
```

3) ***ResNet50***

```
============================================================
Running: ResNet50
C:\Sem5-(TY)\Deep Learning\Deep Learning Labs\Assignment7\venv\Lib\site-packages\keras\src\trainers\epoch_iterator.py:74:
UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Data
set is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
Epoch 1/2
125/125 ━━━━━━━━━━━━━━━━━━━━ 44s 282ms/step - accuracy: 0.7750 - loss: 0.5317 - val_accuracy: 0.8200 - val_loss: 0.3791
Epoch 2/2
125/125 ━━━━━━━━━━━━━━━━━━━━ 33s 267ms/step - accuracy: 0.8710 - loss: 0.2879 - val_accuracy: 0.8300 - val_loss: 0.3661

Final metrics:
Model: ResNet50
Accuracy: 0.8300
Precision: 0.8837
Recall: 0.7600
F1-Score: 0.8172

Classification Report:
              precision    recall  f1-score   support

         Cat       0.79      0.90      0.84       100
         Dog       0.88      0.76      0.82       100

    accuracy                           0.83       200
   macro avg       0.84      0.83      0.83       200
weighted avg       0.84      0.83      0.83       200
```

4) ***EfficientNetB0***

```
============================================================
Running: EfficientNetB0
C:\Sem5-(TY)\Deep Learning\Deep Learning Labs\Assignment7\venv\Lib\site-packages\keras\src\trainers\epoch_iterator.py:74:
UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Data
set is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
Epoch 1/2
125/125 ━━━━━━━━━━━━━━━━━━━━ 31s 135ms/step - accuracy: 0.7600 - loss: 0.5176 - val_accuracy: 0.8950 - val_loss: 0.3319
Epoch 2/2
125/125 ━━━━━━━━━━━━━━━━━━━━ 13s 108ms/step - accuracy: 0.8510 - loss: 0.3497 - val_accuracy: 0.9050 - val_loss: 0.2774

Final metrics:
Model: EfficientNetB0
Accuracy: 0.9050
Precision: 0.8932
Recall: 0.9200
F1-Score: 0.9064

Classification Report:
              precision    recall  f1-score   support

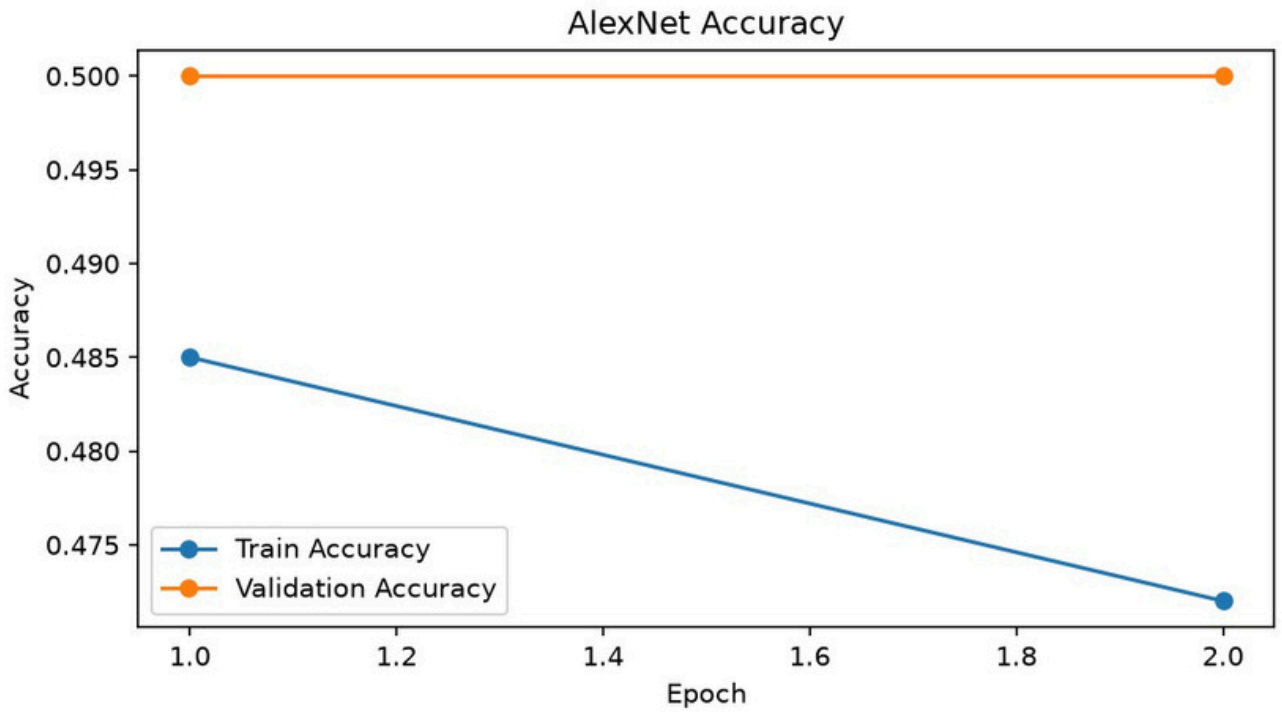
         Cat       0.92      0.89      0.90       100
         Dog       0.89      0.92      0.91       100

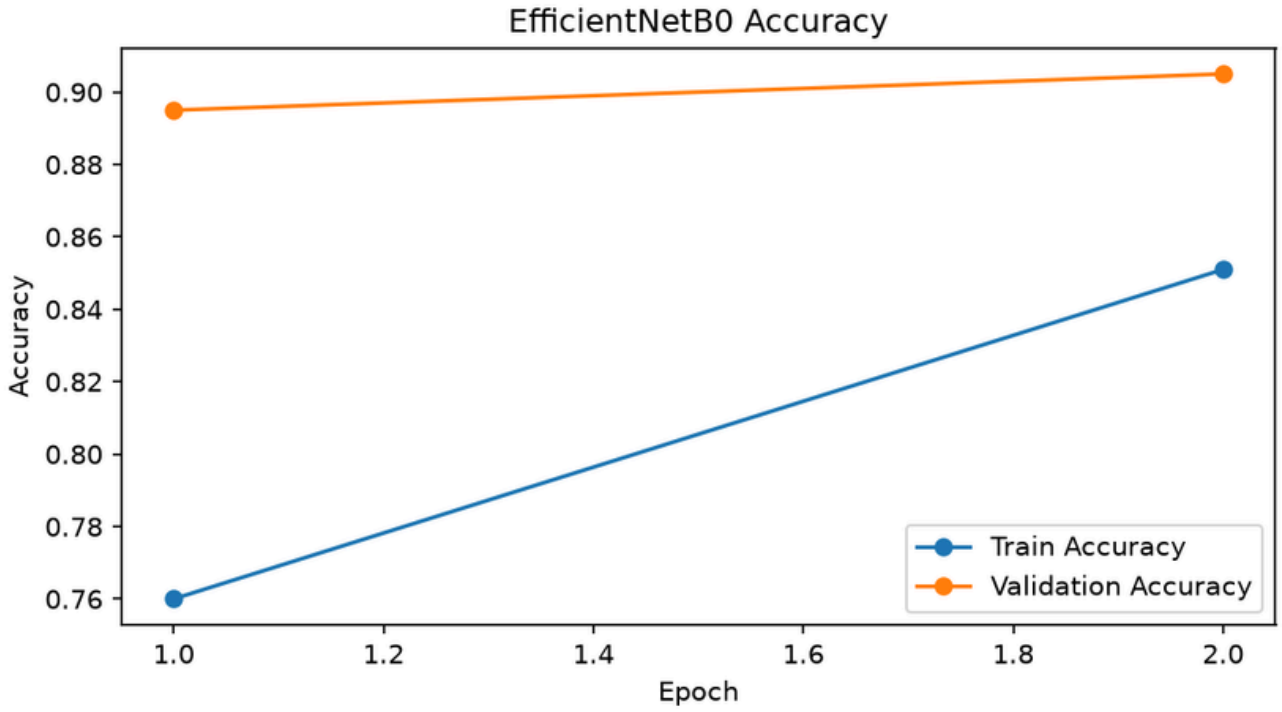
    accuracy                           0.91       200
   macro avg       0.91      0.91      0.90       200
weighted avg       0.91      0.91      0.90       200
```

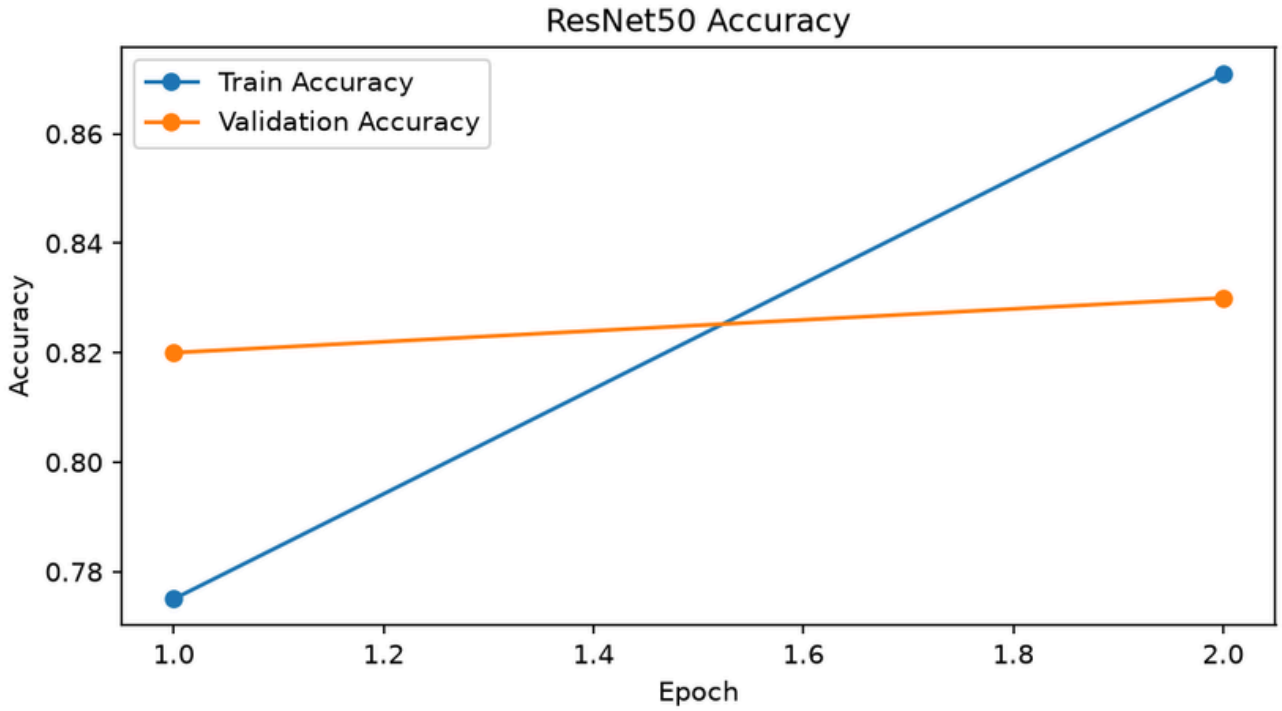
5) Comparison of Models

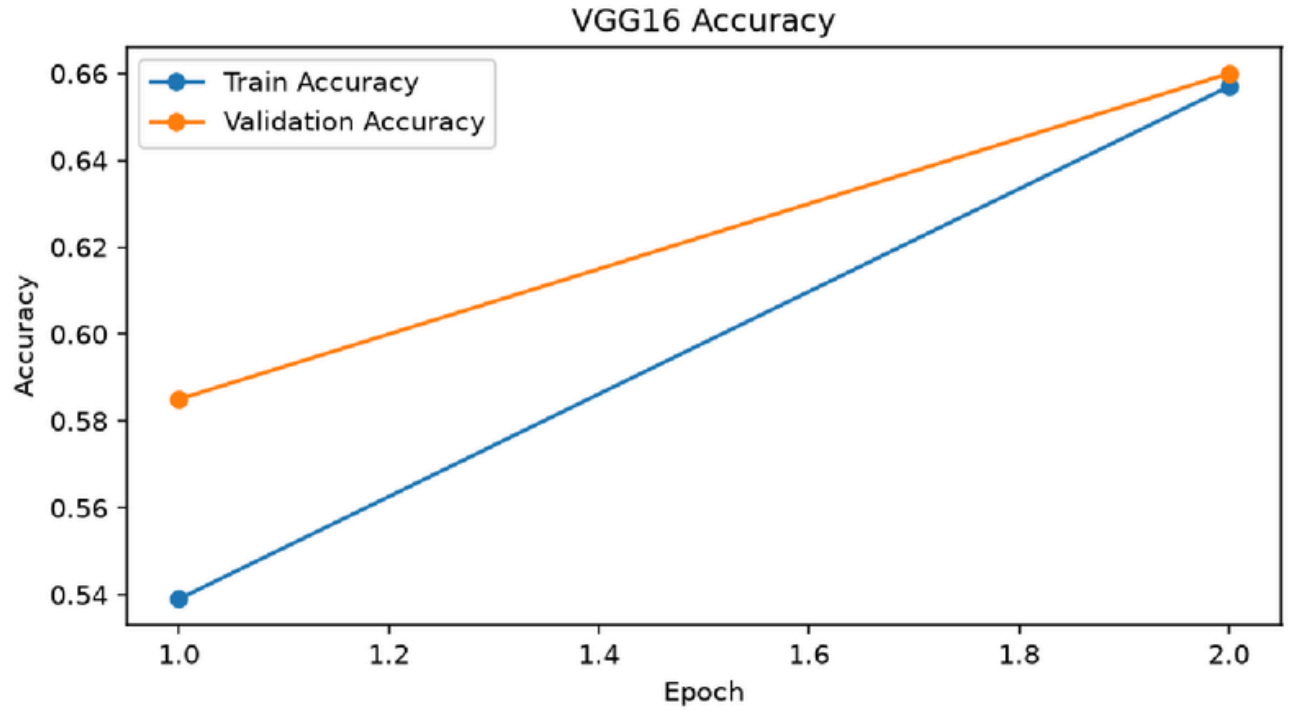
```
============================================================
FINAL COMPARISON
         Model  Accuracy  Precision  Recall  F1-Score
       AlexNet     0.500   0.500000    1.00  0.666667
         VGG16     0.660   0.670213    0.63  0.649485
      ResNet50     0.830   0.883721    0.76  0.817204
EfficientNetB0     0.905   0.893204    0.92  0.906404
```

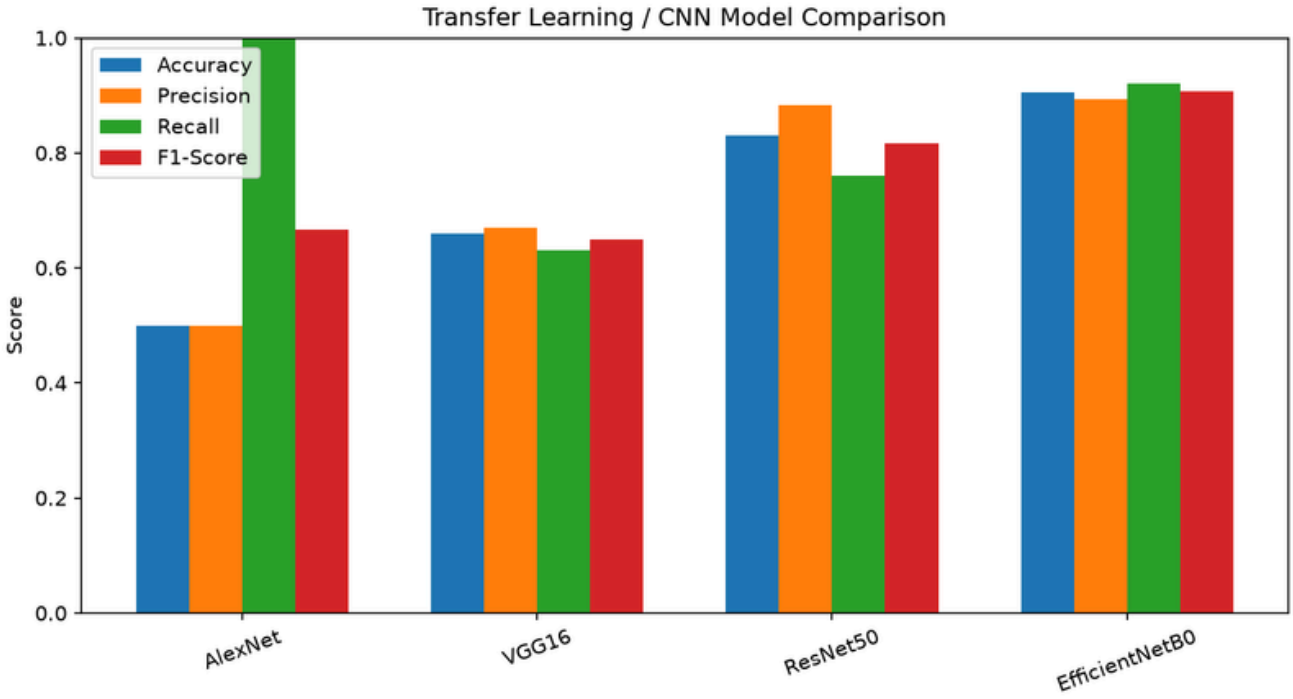
Results :




EfficientNetB0 Accuracy
Accuracy
Epoch
Train Accuracy
Validation Accuracy
0.90
0.88
0.86
0.84
0.82
0.80
0.78
0.76
1.0
1.2
1.4
1.6
1.8
2.0


ResNet50 Accuracy
Accuracy
Epoch
Train Accuracy
Validation Accuracy
0.86
0.84
0.82
0.80
0.78
1.0
1.2
1.4
1.6
1.8
2.0





11. Result Analysis

Themodel havingthe highest validation Accuracy and F1-Score can be considered the best performer for this experiment. Precision and Recall should also be checked to understand the classification behaviour. Because the dataset is intentionally small and the experiment uses only two epochs, the results are primarily for educational comparison rather than a production benchmark.

12. Advantages

Lower training time than training a deep CNN from scratch.
Useful when the target dataset is small.
Pretrained features can provide strong performance.
Freezing layers reduces computation and memory usage.
Different architectures can be compared under the same experimental conditions.

13. Limitations

CIFAR-10 images are originally only 32×32 pixels.
Only a small subset is used for low-memory execution.
Only the classifier is trained for the pretrained models.
AlexNet is represented by an AlexNet-style baseline in standard TensorFlow/Keras because official pretrained AlexNet weights are not provided by tf.keras.applications.
Only two epochs are used, so this is not a fully optimized benchmark.

14. Conclusion

Theimage-classification experiment was implemented using TensorFlow/Keras. VGG16, ResNet50 and EfficientNet-B0 reused ImageNet pretrained features while the final binary classifier was trained for Cat-vs-Dog classification. An AlexNet-style CNN was also implemented as a baseline. Accuracy, Precision, Recall and F1-Score were used for comparison. The actual experimental results determine the final ranking of the models.